

This notebook produces the new experiments requested in review, on the same CNN and classical pipeline as the original study.

Reviewer point	Section here
Dataset characterisation (10 writers, 100% labelled, class distribution)	1

Class-instance explanation (digits 2 and 3, from the 7 primary writers)	1
Writer-independent evaluation (10-writer leave-one-writer-out vs. standard split)	2
Image-resolution justification (why 16x16)	3
Cross-dataset generalisation — systematic factor analysis	4

Run top to bottom. All tables/figures are written to `revision_results/`.

0. Setup

```
import os, json, math, random, warnings
from pathlib import Path
from collections import Counter, OrderedDict
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import torch, torchvision as tv
```

```
from sklearn
from sklearn
from sklearn
from sklearn
```

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, warnings
warnings.filterwarnings("ignore")
plt.rcParams.update({'figure.dpi':110,'axesfig.bbox':'tight'})
DEVICE=torch.device("cuda" if torch.cuda.is_available() else "cpu"); print ("device",DEVICE)

device_cuda = Path(

BASE = Path(r"C:\Users\c8472912\Desktop\VD\paper\digits\final\paper")
CONFIG = {
    "DATA_DIR": str(BASE / "dataset"),
    "HDDA_DIR": str(BASE / "hdda"),
    "RESULTS_DIR": str(BASE / "revision_results"),
    "N_CLASSES": 10,
    "SEEDS": [0, 1], # raise for tighter error bars
    "EPOCHS": 60, "EARLY_STOP_PATIENCE": 8, "BATCH_SIZE": 64,
    "LR": 1e-3, "WEIGHT_DECAY": 1e-4, "DROPOUT": 0.3,
    "AUG_MAX_SHIFT": 2,
}

os.makedirs(CONFIG["RESULTS_DIR"], exist_ok=True)
DIGITS=list(range(10))

def set_seed(s):
    random.seed(s); np.random.seed(s); torch.manual_seed(s); torch.cuda.manual_seed_all(s)
    torch.backends.cudnn.deterministic=True; torch.backends.cudnn.benchmark=False
set_seed(0)

```

1. Dataset characterisation

Loads the dataset from the dir-digit folders and recovers the writer of every image from its filename. Ten writers contributed the 2,560 images; writer identity is available for 100% of the images. Seven writers contributed the full range of digits, which three (morteza, ayasan, ali) contributed smaller balanced sets. The over-representability of digits 2 and 3 comes from the seven primary writers, of which supplied extra 2s and 3s after that confusion was observed.

```

import re
from PIL import Image

# recover the writer of each image from its filename (7 name-prefixed + 3 numbering schemes)
NAMES = ["Mala","aila","alireza","alireza","teyhane","reyhane","dj","dorna",
        "mat","mahsa","sa","hanyeh","nt","negin"]

def writer_of(filename):
    b=re.sub(r'(\.[jpg|jpeg|png|s]*\.','',filename.replace(re.I)); l=w=b.lower()
    for p in ["Mala","alireza","teyhane","dj","ma","sa","nt"]:
        if len(startswith(p) and (len(low)==len(p) or not low[len(p)].isalpha())):
            return NAMES[p]

    if low.startswith("(") and (len(low)==or not low[1].isalpha()): return "morteza" # #- (not ma)
    if re.match(r'^d\d\d\d\d$', b): return "ayasan" # dotted 0.1 ...
    if re.match(r'^0\d\d$', b): return "ali" # zero-pad 01 .. 09
    if re.match(r'^r\d\d \\\(V\d)\??$', b): return "hale" # plain j --> dj
    return None # stray file --> skip

def load_folders(data_dir, image_size=16):
    X,y,w = [],[],[]
    for c in range(10):
        cd = Path(data_dir)/str(c)
        if not cd.exists(): continue
        for fp in sorted(cd.iterdir()):
            if fp.suffix.lower() not in (".jpg",".jpeg",".png"): continue
            w = writer_of(fp.name)
            if w is None: continue
            img = Image.open(fp).convert("L").resize((image_size,image_size))

```

```
X_all, y_all, w_all = load_folders(CO
IMAGE_SIZE = X_all.shape[-1]; N = len
known = np.ones(N, dtype=bool)
```

```
print("per-class counts:", int(k):int(v) for k,v in zip(*np.unravel_index(y_all, return_counts=True)))
named = sorted(set(w_all.tolist()), key=lambda wr:int(w_all==wr).sum())
print("images per writer:", {wr:int(w_all==wr).sum() for wr in named})

mat = pd.DataFrame(0, index=named, columns=DIGITS)
for wr in named:
    for c in DIGITS: mat.loc[wr,c] = int((w_all==wr)&(y_all==c)).sum()
```

```
print(csv_path(CONFIG["RESULTS_DIR"])/f'writer_class_matrix.csv')
print(f'Writer {class_name}, mat')
```

```
Images: 2599, native_size: 16x16, writers: 10 (100% labelled)
Images per class: (0: 187, 1: 196, 2: 253, 3: 333, 4: 192, 5: 186, 6: 187, 7: 182, 8: 182, 9: 191)
Writers per writer: {'alla': 319, 'maha': 314, 'dorna': 310, 'reyhane': 300, 'negin': 300, 'alireza': 300, 'mortez
': 226, 'aylene': 100, 'ali': 90}
```

```
Writer class matrix:
```

	0	1	2	3	4	5	6	7	8	9	total
alla	20	21	70	70	30	23	24	21	20	20	319
maha	10	34	72	72	21	22	22	20	20	20	314
dorna	21	21	71	71	21	21	21	21	21	21	310
reyhane	20	20	70	70	20	20	20	20	20	20	300
negin	20	20	70	70	20	20	20	20	20	20	300
aylene	20	20	70	70	20	20	20	20	20	20	300
alireza	20	20	70	70	20	20	20	20	20	20	300
mortez	37	21	21	21	21	21	21	21	21	21	226
ayshan	10	10	10	10	10	10	10	10	10	10	100
ali	9	9	9	9	9	9	9	9	9	9	90

```
# Figures: per-class counts, samples per writer, writer x class heatmap
fig, ax = plt.subplots(3, 3, figsize=(15, 4))
one = int((y_val+meq).sum())/len(c) # for c in DIGITS
ax[0].bar(DIGITS, ones, color='#2626fa', edgecolor="black")
ax[0].set_ylabel('DIGITS'); ax[0].set_xlabel('digits'); ax[0].set_ylabel("images"); ax[0].set_title("Images per class")
ax[1].axis('tight'); ax[1].text(0, 0, f'one={one} in DIGITS if not in {c,3}'), color='violet', font-size=10, weight='bold', loc='bottomleft')
ax[0].legend(fontsize=8)
swr=writeint(f'wall_time.sum() for wr in named);
ax[1].bar(range(len(swr)), list(swr.values()), color='#faa6fa', edgecolor="black")
ax[1].set_xticks(range(len(swr))); ax[1].set_xticklabels(list(sw.keys(), ), rotation=30, ha='right')
ax[1].set_ylabel("images"); ax[1].set_title("Samples per writer")
ax[2].twinx(ax[1]); ax[2].text(0, 0, f'num={len(DIGITS)} to numpy'), color='blue', aspect='auto', font-size=10, weight='bold', loc='bottomleft')
ax[2].set_xticks(DIGITS); ax[2].set_yticks(range(len(named))); ax[2].set_yticklabels(named)
ax[2].set_xlabel("digits"); ax[2].set_title("Writer x class counts")
```

For `c = 4` in DIGITS:

```
fig, colorbar = plt.subplots(6, 1, int(mat_loc[named['c'], 1]), has="center", va="center", fontsize=6)
fig.colorbar(Im, ax=axs[2], fraction=0.04)
plt.tight_layout(); plt.savefig(Path(CONF1G["RESULTS_DIR"])/"fig_dataset_characterisation.png", dpi=200); plt.show()
```

The figure consists of two plots. The left plot, titled 'Images per class', is a bar chart showing the number of images for each class (0-9). The y-axis ranges from 0 to 500. The bars for classes 0-9 are approximately: 0: 200, 1: 200, 2: 500, 3: 500, 4: 200, 5: 200, 6: 200, 7: 200, 8: 200, 9: 200. A red dashed line indicates the mean of 2.3. The right plot, titled 'Writer x class counts', is a heatmap showing the number of samples per writer (10 writers) and writer x class counts (10 writers x 10 classes). The y-axis ranges from 0 to 300. The x-axis shows the classes 0-9. The color scale ranges from 10 (light blue) to 70 (dark blue). The heatmap shows that the number of samples per writer is relatively uniform across classes, with some variation in the number of samples per writer across classes.

Models and training helpers

Same architecture and protocol as the original

StandardScaler = PCA(0.95). Normalisation/PCA are fit on the training partition of each fold only (no leakage); light pixel-shift augmentation is applied to training images only.

```
class DigitCNN(nn.Module):
    def __init__(self, image_size=16, n_classes=10, dropout=0.3):
        super().__init__()
        self.fc0 = nn.Linear(image_size*image_size, 128)
        self.fc1 = nn.Linear(128, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
```

```

    nn.Conv2d(1,32,3,1,1), nn.BatchNorm2d(32), nn.ReLU(True), nn.MaxPool2d(2),
    nn.Conv2d(32,64,3,1,1), nn.BatchNorm2d(64), nn.ReLU(True), nn.MaxPool2d(2))

    with torch.no_grad():
        flat=self.features(torch.zeros(1,1,image_size,image_size).null())
        self.classifier=nn.Sequential(*[nn.Flatten(), nn.Dropout(dropout),
        nn.Linear(flat,128), nn.ReLU(True), nn.Dropout(dropout), nn.Linear(128,n_classes)])
    def forward(self,x): return self.classifier(self.features(x))

class DS(Dataset):
    def __init__(self,X,y,mean,std,aug=False,ns=2):
        self.X,self.y,self.mean,self.std,self.aug,self.ms,self.Xy,self.mean,std,aug,ns
    def __len__(self): return len(self.y)
    def __getitem__(self, img):
        dx=np.random.randint(-(self.ms,self.ms*1))
        dy=np.random.randint(-(self.ms,self.ms*1))
        imgp=roll(img,(dy,dx),axis=(1,2))
        if dy>0: img[1:,dy:,1:]<=0
        elif dy<0: img[1:,-dy:,1:]<=0
        if dx>0: img[:,1:,dx:]<=0
        elif dx<0: img[:,1:,-dx:]<=0
        return img
    def __getitem__(self,i):
        imgp=self.X[i].copy()
        if self.aug: imgp=self.__getitem__(img)
        return torch.from_numpy((imgp-self.mean)/(self.std).float(), 1, int(self.y[i]))

def train_cnn(Xtr,ytr,Xva,yva,cfg,seed,image_size):
    set_seed(seed); mean,std=float(Xtr.mean()),float(Xtr.std()+1e-7)
    tl=DataLoader(DS(Xtr,ytr,mean,std,True,cfg["AUG_MAX_SHIFT"]),batch_size=cfg["BATCH_SIZE"],shuffle=True)
    vl=DataLoader(DS(Xva,yva,mean,std,batch_size=256))
    m=DigitCNN(image_size,cfg["N_CLASSES"],cfg["DROPOUT"],to_DEVICE)
    opt=torch.optim.Adam(m.parameters()),lr=cfg["LR"],weight_decay=cfg["WEIGHT_DECAY"]; crit=nn.CrossEntropyLoss()
    best,b,walt=1e9,None,0
    for fup in range(cfg["EPOCHS"]):
        m.train()
        for xb,yb in tl:
            xb,yb=to_DEVICE(xb,yb); opt.zero_grad(); crit(m(xb,yb).backward()); opt.step()
        m.eval(); vloss=0.0
        with torch.no_grad():
            for xb,yb in vl:
                vloss+=crit(m(xb,to_DEVICE(yb),to_DEVICE(yb)).item()*len(xb)
                vloss/len(yb)
            if vloss<best-1e-4: best,b,walt=vloss,k,v.cpu().clone()
        for k,v in m.state_dict().items(),0
        else:
            walt+=1
            if walt==cfg["EARLY_STOP_PATIENCE"]: break
        m.load_state_dict(bst); return m,(mean,std)

@torch.no_grad()
def predict_cnn(m,X,y,st):
    dl=DataLoader(DS(X,y,st[0],st[1],batch_size=256),m.eval(),out=[])
    for xb,_ in dl: out.append(m(xb,to_DEVICE(yb)).argmax(1).cpu().numpy())
    return np.concatenate(out)

def make_classifiers():
    return OrderedDict({
        "KNN":KNeighborsClassifier(m_neighbors=3),
        "SVM":SVC(kernel="poly",degree=2,gamma=0.01,C=10),
        "Naive Bayes":GaussianNB(),
        "Decision Tree":DecisionTreeClassifier(max_depth=10,min_samples_leaf=3,min_samples_split=7,random_state=0),
        "Random Forest":RandomForestClassifier(n_estimators=300,random_state=0)})

def make_pipe(cfg): return Pipeline(("*",StandardScaler()),("P",PCA(0.95,random_state=0)),("t",clf))
MODELS=["CNN"];klist=(make_classifiers().keys())

def fit_predict_all(Xtr,ytr,Xte,yte,cfg,seed,image_size):
    preds=[]
    # internal validation split (from the training partition only) for CNN early stopping
    it,train=train_split(np.arange(len(ytr)),batch_size=0.15,cfg["N_CLASSES"])/len(ytr),stratify=ytr,random_state=seed)
    m=fit_train_cnn(Xtr[ytr[itr]],ytr[itr],ytr[te],cfg,seed,image_size)
    preds["CNN"]=predict_cnn(m,Xte,yte,st)
    Xtrf,Xtef=Xtr.reshape(len(Xtr),-1),Xte.reshape(len(Xte),-1)
    for name,clf in make_classifiers().items():
        preds[name]=make_pipe(clf).fit(Xtrf,ytr).predict(Xtef)
    return preds

2. Writer-dependent vs. writer-independent protocol

The key new experiment. We restrict to the writers who contributed all ten digits and compare two protocols on the same image pool, differing only in how digits are formed:



- Writer-dependent (WD): stratified k-fold – samples from a writer may appear in both train and test.
- Writer-independent (WI): leave-one-writer-out – the test writer is never seen in training.



The gap WD – WI measures how much accuracy depends on having seen the test writer.

main_writers=[w for w in named if int((w[all==wz].sum())/50)] # all writers with a usable set
mask=np.isin(W.all(), main_writers)
Xw,yw,qw=X[W.all(mask),y[all(mask),y[all(mask)]
print "writer-independent pool:" ,len(yw),"Images from",len(main_writers),"writers",main_writers)

def macro (yt,ypl): return accuracy_score(yt,ypl), precision_recall_fscore_support(yt,ypl,average="macro",zero_division=0)[2]

# ND: stratified k-fold (k = number of writers, for comparability), repeated over seeds
rows=[]
K=len(main_writers)
wL_accs=[] for s in MODELS:
    for seed in CONFIG["SEEDS"]:
        skf=StratifiedFold(K,shuffle=True,random_state=seed)

```

```
# WI: leave-one-writer-out
wi_acc=(m:[] for m in MODELS)
logo=LeaveOneGroupOut()
```

```
pr=fit_predict_all(Xw[ttr],yw[ttr],yw[tte],CONFIG.CONFIG["SEEDS"][0],IMAGE_SIZE)
for m in MODELS: w1_acc[m].append(macro(yw[tte],pr[m])[0])

tbl=[]
for m in MODELS:
    wd,wi=np.array(wd_acc[m]),np.array(wi_acc[m])
    tbl.append(["model":m,"WD_acc":wd.mean()*100,"WD_std":wd.std()*100,
```

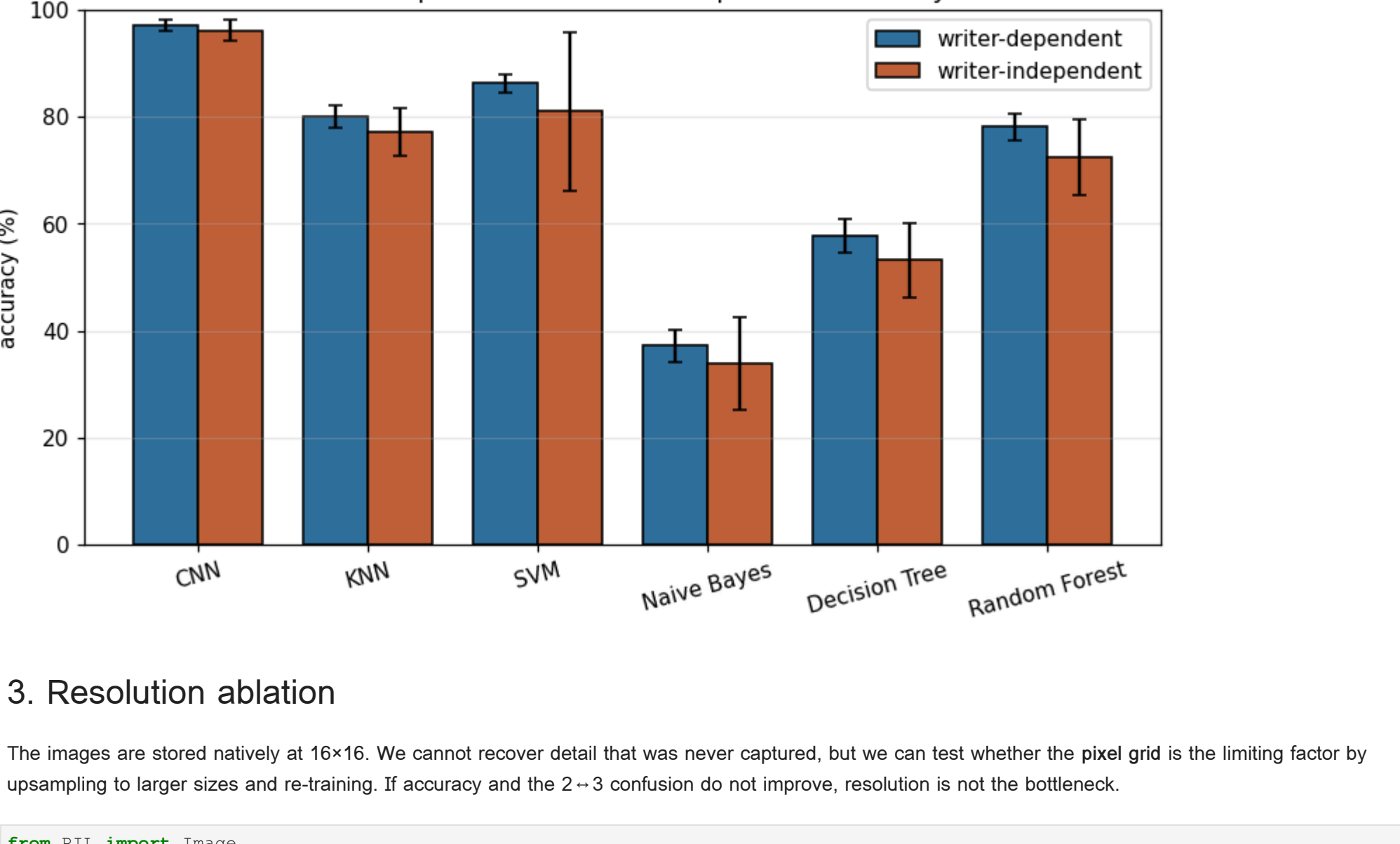
- ```
wdwi=pd.DataFrame(tb); wdwi.to_csv(Path(CONFIG["RESULTS_DIR"])/"writer_dependent_v")
print(wdwi.round(2).to_string(index=False))

writer-independent pool: 2559 images from 10 writers: ['aila', 'maha', 'dorna', 'rey']
```

```
File "C:\Users\c8472912\AppData\Local\anaconda3\Lib\site-packages\jobli
_physical_cores
cpu_info = subprocess.run(
```

```
File "C:\Users\c8472912\AppData\Local\anaconda3\Lib\subprocess.py", line 548, in run
with Popen(*popenargs, **kwargs) as process:
.....
File "C:\Users\c8472912\AppData\Local\anaconda3\Lib\subprocess.py", line 1026, in __init__
self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\c8472912\AppData\Local\anaconda3\Lib\subprocess.py", line 1538, in _execute_child
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
.....
model WD_acc MD_std WI_acc WI_std gap
CNN 97.01 1.01 96.11 1.98 0.90
SVM 80.01 2.19 77.13 4.50 2.88
SVM 86.19 1.64 81.02 14.85 5.16
Naive Bayes 37.12 2.89 33.86 8.72 3.27
Decision Tree 57.72 3.15 53.13 7.07 4.58
Random Forest 78.06 2.47 72.37 7.15 5.69

fig,ax=plt.subplots(figsize=(9,4.5)); xmpg_array=len(MODELS); wbar=0.38
ax.bar(k=[wbar,z],wdwi_sel_index'(model').loc[m,'WD_acc'] for m in MODELS),wbar,
yerr=(wdwi_sel_index'(model').loc[m,'MD_std'] for m in MODELS),capsize=3,label="writer-dependent",color="#266f9f",edgeco
ax.bar(k=[wbar,z],wdwi_sel_index'(model').loc[m,'WI_acc'] for m in MODELS),wbar,
yerr=(wdwi_sel_index'(model').loc[m,'WI_std'] for m in MODELS),capsize=3,label="writer-independent",color="#c0603a",edgeco
ax.set_ticks(x); ax.set_ticklabels(MODELS,rotation=15); ax.set_ylabel("Accuracy (%)"); ax.set_ylim(0,100)
ax.set_title("Writer-dependent vs. writer-independent accuracy"); ax.legend(); ax.grid(xaxis="y",alpha=.3)
plt.savefig(Path(CONFIG['RESULTS_DIR']+"/fig/writer_independent.png",dpi=200); plt.show()
```



```
for i in range(len(X)):
 out[i,0]=np.asarray(In[i,0])
return out
```

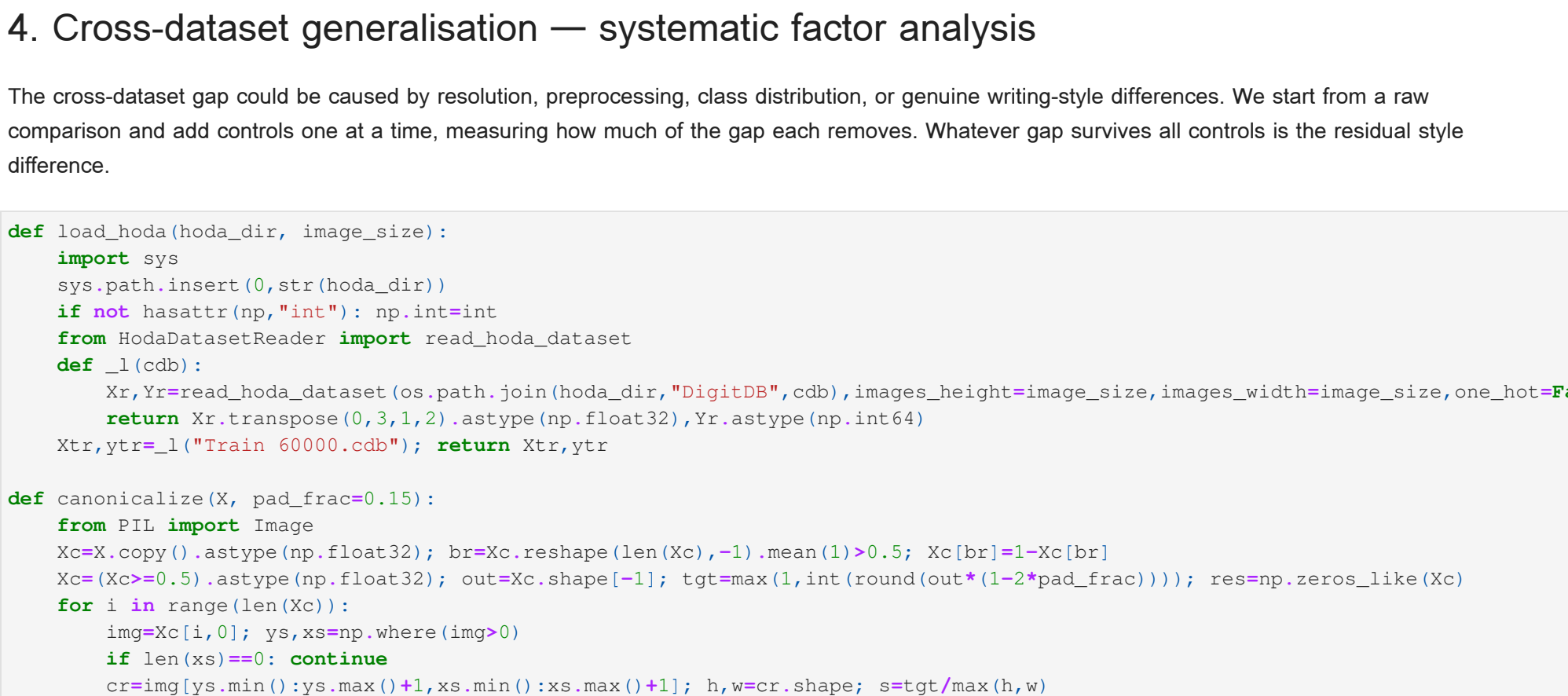
```
test_rows =
dev, test=train_test_split(np.arange(N), test_size=0.2, stratify=y_all, random_state=0)
for size in [16, 20, 24, 28]:
 X_all = y_all
 if size==16 else else_stack(X_all, size)
 accs=[]; conf23=[]
 for seed in CONFIG["SEEDS"]:
 prf=prf_predict_all(Xr[dev], y_all[dev], Xr[test], y_all[test], CONFIG, seed, size) ["CNS"]
```

```

cm=confusion_matrix(y_all[test],pr_labels=DIGITS.astype(float))
cmcm=cm.sum(1,keeping=True,cip=(min))
conf23.append((cm[2,3]+cm[3,2])/2)

res_rows.append(("resolution"+"%f"%(size)x(size),"cmn_acc":np.mean(accs)*100,"cmn_std":np.std(accs)*100,
 "confusion_2_3":np.mean(conf23)*100))
resdf=pd.DataFrame(res_rows); resdf.to_csv(Path(CONFIG+"RESULTS_DIR"))/*resolution_ablation.cmn,index=False)
print(resdf.round(2).to_string(index=False))
fig,ax=plt.subplots(figsize=(7,4)); xs=[16,20,24,28]
ax.errorbar(xs,resdf["cmn_acc"],yerrresdf["cmn_std"],marker="o",label="CNN accuracy",color="#2f6f9f")
ax.twinx(min(1); ax2=plt.gca().twinx(); ax2.set_ylabel("2+3 confusion (%)",color="#c0003a",label="2+3 confusion (%)")
ax.set_xlabel("input resolution (upsampled)"); ax.set_ylabel("accuracy (%)"); ax2.set_ylabel("2+3 confusion (%)")
ax.set_title("Effect of (upsampled) resolution"); ax.set_xticks(xs)
lines=ax.get_lines()+ax2.get_lines(); ax.legend(lines,1.get_label() for i in lines),loc="center right")
plt.savefig(Path(CONFIG+"RESULTS_DIR"))/*fig_resolution.png",dpi=200); plt.show()

```



```

 return res
def balance(X, y, n, seed=0):
 rng=np.random.RandomState(seed); idx=[]

```

```

 ci=np.where(y==c)[0]; idx.append(rng.choice(ci,min(n,len(ci)),replace=len(ci)<n))
 idx=np.concatenate(idx); rng.shuffle(idx); return X[idx],y[idx]

try:
 Xh,yh=load_hoda(CONFIG["HODA_DIR"],IMAGE_SIZE)
 Xh,yh=balance(Xh,yh,600,0) # subsample HODA for speed
 Xo,yo=X_all,y_all

```

```

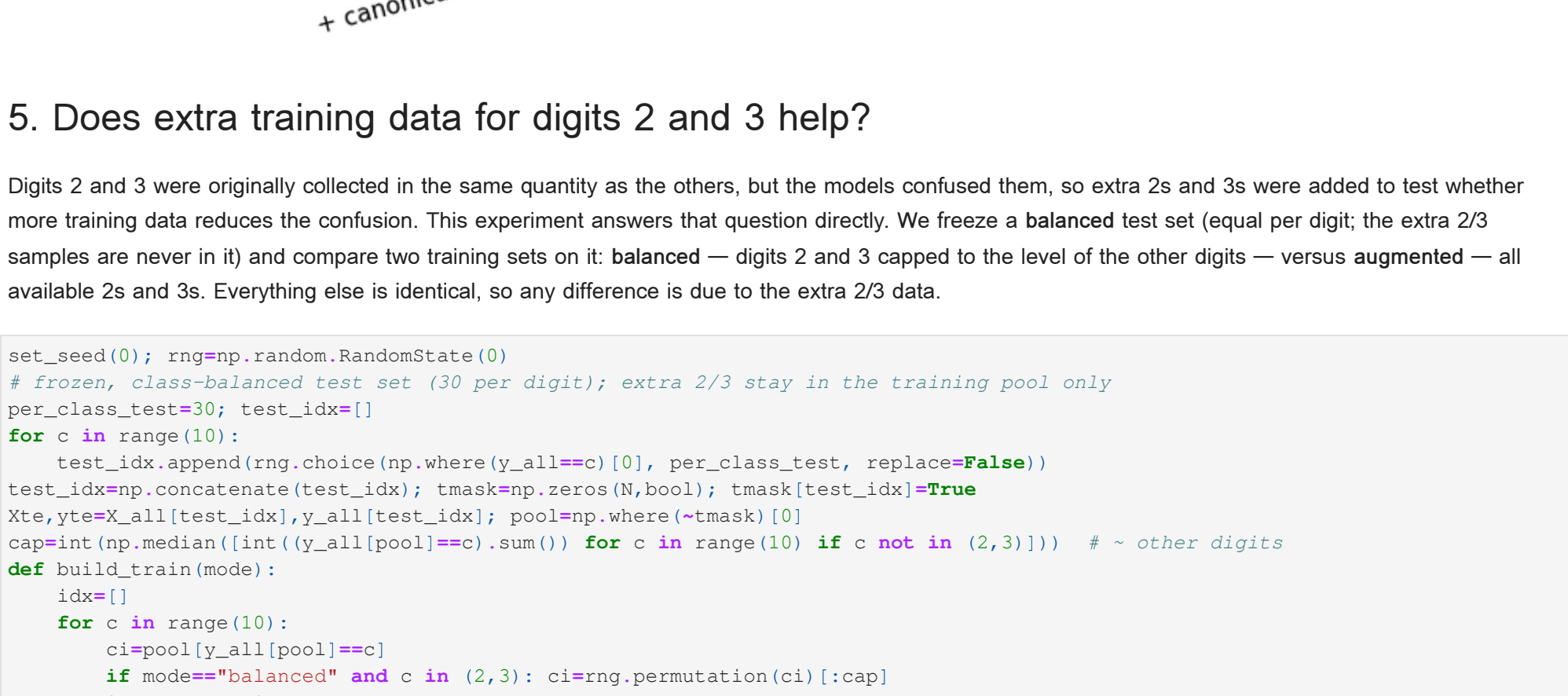
itr,iv=train_test_split(np.arange(len(ytr)),test_size=0.15, stratify=ytr, random_state=0)
st=st_train_cnn(Xtr[itr],ytr[itr],Xtr[iv],ytr[iv],CONFIG,0,IMAGE_SIZE)
return accuracy_score(yte,predicit_cnn(Xt,Xte,yte,st))
stages=[]
stage 0: raw (same 16x16 already; different preprocessing & class balance)
stages.append(("raw",Xo,yo,Xh,yh))
stage 1: canonical preprocessing on both
Xo_c,Xh_c=canonicalize(Xo),canonicalize(Xh)
stages.append(("canonical preprocessing",Xo_c,yo,Xh_c,yh))
stage 2: + class balance (both balanced to 150/class)
Xo_b,yo_b,balance=(Xo_c,yo,150,1); Xh_b,yh_b,balance=(Xh_c,yh,150,1)
rows=[]
for name,Xo_s,yo_s,Xh_s,yh_s in stages:
 o2h=etrain(Xo_s,yo_s,Xh_s,yh_s); h2o=etrain(Xo_s,yh_s,Xo_s,yo_s)
 rows.append(("stage":name,"ours-HODA":o2h*100,"HODA-ours":h2o*100,"asymmetry":(o2h-h2o)*100))
 o2h=etrain(Xo_b,yo_b,Xh_b,yh_b); h2o=etrain(Xh_b,yh_b,Xo_b,yo_b)
 rows.append(("stage": "class balance","ours-HODA":o2h*100,"HODA-ours":h2o*100,"asymmetry":(o2h-h2o)*100))
crossed.DataFrame(rows); cross.to_csv(Path(CONFIG["RESULTS_DIR"])/"cross_dataset_factors.csv",index=False)
fig,cross=plt.subplots(figsize=(8,4.5)); xsmpp=arange(len(cross))
ax,plot=cross["ours-HODA"],";",o",label="train ours -> test HODA",color="#2f6f9f")
ax,plot=cross["HODA-ours"],";",o",label="train HODA - test ours",color="#c0603a")
ax.set_xticks(xsmpp); ax.set_xticklabels(cross["stage"],rotation=15,hw="right")
ax.set_ylabel("accuracy (%)"); ax.set_title("Cross-dataset accuracy as factors are controlled")
ax.legend(); ax.grid(alpha=.3)
plt.savefig(Path(CONFIG["RESULTS_DIR"])/"fig_cross_dataset_factors.png",dpi=200); plt.show()
except Exception as e:
 print("HODA cross-dataset step skipped:", e)

```

| stage                   | ours-HODA | HODA-ours | asymmetry |
|-------------------------|-----------|-----------|-----------|
| raw                     | 31.23     | 8.36      | 22.87     |
| canonical preprocessing | 96.05     | 83.78     | 12.27     |
| + class balance         | 93.13     | 85.67     | 7.47      |

**Cross-dataset accuracy as factors are controlled**

| stage                   | train ours -> test HODA (%) | train HODA -> test ours (%) |
|-------------------------|-----------------------------|-----------------------------|
| raw                     | 31.23                       | 8.36                        |
| canonical preprocessing | 96.05                       | 83.78                       |
| + class balance         | 93.13                       | 85.67                       |



```
rows=[]
for mode in ["balanced","augmented"]:
 accs,f2,f3,c23=[],[],[],[]
```

```
Xtr,ytr=ubuild_train(mode)
pr=fit_predict_all(Xtr,ytr,Xte,yte,CONFIG,seed,IMAGE_SIZE)*["CNN*"]
accs.append(accuracy_score(yte,pr))
f2,f3=precision_recall_fscore_support(yte,pr,labels=DIGITS,zero_division=0)
f2.append(f1[2]),f3.append(f1[3])
cm=confusion_matrix(yte,pr,labels=DIGITS).astype(float); cmn=cm/cm.sum(1,keepdims=True).clip(min=1)
c23.append((cmn[2,3]+cmn[3,2])/2)
```

```
rows.append(["training", mode, "n_2and3_each", n23, "overall_acc", inp.mean(accs)*100,
 "fl_digit2", inp.mean(f2)*100, "fl_digit3", inp.mean(f3)*100, "confusion_2to3", inp.mean(c23)*100])
imbpd.DataFrame(rows).to_csv(Path(CONFIG["RESULTS_DIR"])/f"class_imbalance_ablation.csv", index=False)
print(imb.round(2).to_string(index=False))

fig, ax=plt.subplots(figsize=(7.5,4)); x=np.arange(3); wbar=0.38
b=imb[imb.training=="balanced"]; iloc[0]; ax=imb[imb.training=="augmented"]; iloc[0]
ax.bar(x#wbar/2, [b["fl_digit2"], b["fl_digit3"], b["confusion_2to3"]], wbar, label=f"balanced (2,3 = {int(b['n_2and3_each'])} each)");
ax.bar(x#wbar/2, [a["fl_digit2"], a["fl_digit3"], a["confusion_2to3"]], wbar, label=f"augmented (2,3 = {int(a['n_2and3_each'])} each)");
ax.set_xticks(x); ax.set_xticklabels(["fl digit 2", "fl digit 3", "2+3 confusion"]); ax.set_ylabel("F1")
ax.set_title("Effect of extra 2/3 training data (CNN, same frozen test)"); ax.legend(title="#"); ax.grid(axis="xy", alpha=.3)
plt.savefig(Path(CONFIG["RESULTS_DIR"])/f"fig_class_imbalance.png", dpi=200); plt.show()

training n_2and3_each overall_acc fl_digit2 fl_digit3 confusion_2to3
balanced 156 97.33 90.76 93.55 6.67
augmented 503 97.67 94.92 96.77 3.33
```

**Effect of extra 2/3 training data (CNN, same frozen test)**

| Category      | balanced (2,3 = 156 each) | augmented (2,3 = 503 each) |
|---------------|---------------------------|----------------------------|
| F1 digit 2    | 90.76                     | 94.92                      |
| F1 digit 3    | 93.55                     | 96.77                      |
| 2+3 confusion | 6.67                      | 3.33                       |

## 6. Save a summary

All tables and figures are in `revision_results/`. Use them to write the revised paper:

- Table `writer_class_matrix.csv` and Table characterisation – dataset section.
- Table `writer_dependent_vs_independent.csv` and Fig. writer-independent – the new centrepiece.
- Table `resolution_ablation.csv` and Fig. resolution – resolution justification.
- Table `cross_dataset_factors.csv` and Fig. cross-dataset – systematic cross-dataset analysis.

```
summary=["n_images":int(N), "image_size":int(IMAGE_SIZE),
 "n_classes":int(n_classes), "confusion_matrices":np.zeros((n_classes, n_classes))]
```



```
"6": 187,
"7": 182,
"8": 182,
"9": 181
```

